☁️ Cloud Architecture & Infrastructure

Week 5 • Day 2 • Technical Infrastructure & Modeling

TPM Academy

🎯 Day 2 Objectives

- Make the **region + AZ** choice in customer + compliance + cost terms
- Default to **managed services**; defend any self-managed exception
- Take a **multi-tenancy stance** tied to customer evidence
- Choose a **network boundary** with revisit triggers
- Add a **ROM cost table** — catch 2x surprises before they ship

🕑 Today's Shape

Clock	Block	Outcome
09:00 – 09:15	Opening	Pin TMD §1 on the wall
09:15 – 10:30	Block 1 + Activity 1	Region + AZ stance
10:45 – 12:00	Block 2 + Activity 2	Managed vs self per component
13:00 – 14:15	Block 3 + Activity 3	Multi-tenancy + network boundary
14:30 – 16:00	Block 4 + Activity 4 + Wrap	ROM cost + AI sanity check

Block 1 — Region + AZ

Where the customer's data lives matters

📚 The Cloud Vocabulary Card

Term	What it means
Region	A geographic cloud location (us-east-1)
AZ	Failure-isolated facility within a region
Multi-region	Across regions; survives whole-region failure
Managed service	Cloud provider runs it (RDS, S3, SQS)
Self-managed	You run it on raw compute
Multi-tenant	One deployment serves all customers
Single-tenant	Each customer gets their own
Edge / CDN	Static content cached close to users

🎯 The Five Decisions

1. **Region(s)** — which?
2. **Multi-AZ** — single, multi-AZ, or multi-region?
3. **Managed vs self-managed** — for each component
4. **Multi-tenancy** — shared / pooled / dedicated / hybrid
5. **Network boundary** — public, VPN, Direct Connect

Each has a **cost dimension** and a **risk dimension**. Surface both.

📝 Worked Region + AZ Stance

```
**Region:** us-east-1 (primary), us-west-2 (warm-DR).
- Customer base: 92% US; 8% Canada (cross-region OK; +80ms)
- Compliance: SOC 2 audit retention requires named region

<p><strong>AZ stance:</strong> Multi-AZ active-active for stateful tiers
(Postgres primary + 2 read replicas across 3 AZs).</p>
```

```
<strong>Trade-off — multi-region active-active vs warm-DR:</strong>
<strong>Choice:</strong> Warm-DR (Option A).
<strong>Why:</strong> Active-active doubles cost and adds cross-region
consistency complexity; multi-AZ within us-east-1
satisfies our 99.5% availability SLO.
<strong>Accepted cost:</strong> Full us-east-1 outage breaches SLO ~30 min.
<strong>Revisit trigger:</strong> Second multi-hour us-east-1 outage in
12 months, OR availability SLO tightens to 99.9%+.
```

👥 Activity 1 — Region + AZ

Triads • 35 minutes

- 5 min: pull customer location facts (PRD §2)
- 5 min: pull compliance facts (TCD §3)
- 10 min: decide region(s)
- 10 min: decide multi-AZ stance
- 5 min: capture trade-off (Week-4 template)

⏱️ 5 + 5 + 10 + 10 + 5

Block 2 — Managed vs Self

Default to managed; defend exceptions

❓ The Four-Question Test

Default: **use the managed service**. Reasons to self-manage:

1. Do failure modes match our needs?
2. Is cost reasonable at expected scale?
3. Does compliance allow it?
4. Do we have a specific reason to control this?

> **"We want full control"** is a smell. Push: what specific decision do you need that the managed service forecloses?

📝 The Managed/Self Table

Component	Managed option	Choice	Why
Postgres	AWS RDS	Managed	Default; team ops capacity finite
Kafka	AWS MSK	Managed	Same
Object storage	S3	Managed	Industry-standard durability
Custom AI service	None	Self-managed	No managed equivalent
Load balancer	ALB	Managed	Trivial to use; standard

👥 Activity 2 — Managed vs Self

Triads • 40 minutes

- 5 min: list each container (TCD §2)
- 10 min: identify managed-service option per container
- 15 min: managed vs self per container (4-question test)
- 10 min: capture the table

⏱️ 5 + 10 + 15 + 10

Block 3 — Tenancy + Network

Stakeholder-touching decisions

👨‍👩‍👦 Multi-Tenancy Spectrum

Stance	Architecture	Pros	Cons
Shared	One deployment, all tenants	Cheap	Noisy neighbor
Pooled	One deployment + per-tenant limits	Cheap-ish; isolated	Tenant-aware code
Dedicated	One deployment per tenant	Strongest isolation	Expensive
Hybrid	Most pooled; some dedicated	Match cost to value	Two patterns

Most B2B SaaS at our stage: pooled with per-tenant rate limits (revisit Week 4 Day 4).

🔗 Network Boundary

Public HTTPS

Default for B2B SaaS APIs. Standard TLS; nothing else.

Customer-VPN-only

Some enterprise contracts; customer comes through their VPN tunnel.

Direct Connect / PrivateLink

Large enterprise; regulated industries; dedicated network paths.

Don't volunteer for VPN-only. It triples ops complexity. Only choose if a customer requires it.

👥 Activity 3 — Tenancy + Network

Triads • 40 minutes

- 10 min: customer tenancy expectations (PRD §1)
- 10 min: pick stance with revisit trigger
- 15 min: network boundary stance
- 5 min: stakeholder implications (add to TCD §6 if missing)

⏱️ 10 + 10 + 15 + 5

Block 4 — Cost + AI Sanity

Catch 2x surprises before they ship

💰 ROM Cost Table

Component	Stance	Cost / month	Driver
Postgres (RDS)	Multi-AZ db.m5.large	400–700	Hours; storage; IOPS
Read replica	Single-AZ	200–350	Hours
Kafka (MSK)	3-broker m5.large	700–1100	Brokers; storage; egress
S3 (audit)	Standard tier	$50/TB-month	Storage; PUT/GET
ALB	Standard	$25 + traffic	Hours + LCUs
Egress bandwidth		50–150	$/GB out

Goal: rough order of magnitude. Engineering will refine. The TPM job: catch the 2x surprise.

AI Sanity Check

```
Role: SRE reviewing a feature's cloud topology.
Context: <paste tmd="" §2="" sections="" —="" region,="" az,="" managed="" services,="" tenancy,="" network="" boundary,="" rom="" c
Task: Identify the top 3 risks in this topology that could
surprise the team within 6 months.
Constraints:
- Be specific to the configurations described
- For each: scenario, likelihood (L/M/H), suggested mitigation
- Do not invent platform features
Format: Numbered — Risk / Scenario / Likelihood / Mitigation.</paste>
```

👥 Activity 4 — Cost + AI Sanity

Triads • 45 minutes • Wrap

- 15 min: ROM cost table
- 10 min: AI sanity check (3 risks)
- 10 min: adopt / defer / reject
- 10 min: polish §2 + provenance note

⏱️ 15 + 10 + 10 + 10 · 15-min wrap

🎉 Day 2 Wrap

What we built

- Region + AZ stance
- Managed/self table
- Multi-tenancy + network boundary
- ROM cost table
- TMD §2 drafted

What opens tomorrow

- **REST & SOAP API fundamentals**
- Resource modeling, idempotency, errors
- When SOAP is still right

Bring to Day 3: TMD §§1–2 + your data model. APIs ride on top of both tomorrow.

💬 Questions & Reflection

Where does your topology assume the team has expertise it might not have in 12 months?